Computación gráfica y ambientes virtuales (0494)

Práctico 07

Objetivo

El objetivo es agregar a la escena del simulador el terreno por el cual moverse. Para ello se utilizará una malla poligonal que represente una topografía con relieve, generada por programa a partir de una fuente de alturas y dibujada con *eliminación de superficies ocultas* activada. Hasta acá todas las mallas usadas en el proyecto se escribieron a mano (cubo) o salieron de una primitiva paramétrica (cono, cilindro); ésta es la primera que se genera a partir de datos externos y el objeto físico que va a representar (topografía del terreno) requiere que sea lo bastante grande como para estudiar cómo se decide su construcción: cuántos vértices, en qué orden, la escala y qué sucede cuando la aeronave llega al borde de la malla.

Punto de partida

La escena del Práctico 06 o la del 05 si el modelo de dinámica de vuelo (FDM) todavía no está integrado. Nada de este práctico requiere que el FDM esté funcionando: el terreno, las normales y el sistema de cámaras se pueden construir sin que la aeronave se esté moviendo. Lo único que conviene tener funcionando es alguna forma de desplazarse por la escena —la cámara orbital alcanza— para poder mirar el terreno desde distintos lugares y verificar las condiciones extremas (qué sucede cuando llegamos al borde).

Parte 1 — La malla poligonal para el terreno

Hay que generar una malla poligonal de $N \times N$ **celdas** sobre el plano horizontal. La misma se genera por programa por medio de una *grilla regular de nodos* que corresponden a los vértices de las celdas. La generación se puede implementar como una función libre que recibe el tamaño de la grilla (N), la separación entre vértices y la fuente para leer las alturas; lo que devuelve son dos listas: una con los vértices y otra con los índices. De esta forma es posible cambiar, luego, el relieve sin tocar la posición de los nodos de la grilla y el orden en que se tienen que dibujar.

```
// Los vertices. Ojo con el limite: <= N porque hay N+1 vertices por lado.
for (unsigned i = 0; i <= N; ++i){
    for (unsigned j = 0; j <= N; ++j){
        vertices.push_back({ j * paso,  altura(i, j),  i * paso });
    }
}

// Los indices. Aca el limite es < N porque se recorren CELDAS.
const unsigned fila = N + 1;                 // ancho en VERTICES, no en celdas
auto idx = [fila](unsigned i, unsigned j) { return i * fila + j; };

for (unsigned i = 0; i < N; ++i){
    for (unsigned j = 0; j < N; ++j) {
        const unsigned a = idx(i, j),     b = idx(i,   j+1);
        const unsigned d = idx(i+1, j),   c = idx(i+1, j+1);
        indices.insert(indices.end(), {a, d, c});
        indices.insert(indices.end(), {a, c, b});
    }
}
```

Malla indexada

Si hay N celdas por lado entonces hay $N + 1$ vértices por lado. Para $N = 200$ se generan **40.401 vértices** y la malla poligonal resultante tiene **80.000 triángulos**. Como cada vértice interior lo comparten seis triángulos, sin indexar habría que listar un vértice por cada referencia, lo que daría un total de 240.000 en lugar de 40.401.

A diferencia de la malla del cubo del Práctico 02 —donde cada esquina pertenece a tres caras de distinto color y por eso no se podía compartir la tupla— acá los seis triángulos que tocan un vértice interior pueden usar **la misma tupla de atributos**. La condición para compartir no es geométrica sino de sombreado (Unidad IX - Iluminación): en el cubo la misma esquina necesita una normal *distinta* por cara, y por eso hay que repetirla tres veces; en el terreno se decide usar **una sola normal por vértice** —la que corresponde a una superficie suave, que se calcula en la Parte 2— y entonces la tupla es única.

Comparativamente, indexar da 5,94 veces menos vértices, pero en memoria la mejora es de 2,99 veces porque el buffer de índices termina con un tamaño que es casi el mismo que el de vértices.

Parte 2 — Las alturas, las normales y las escalas

El modelo de alturas

Una estrategia para deformar la grilla creada en la Parte 1 y que tenga la apariencia de un terreno es utilizar un *heightmap* que es una grilla de números: un valor de altura por nodo, sin datos de geometría ni triángulos. Estos datos pueden venir en distintos formatos como ser una imagen en escala de grises —un canal por píxel—, un archivo de texto (con datos topográficos reales, por ejemplo) o de una función evaluada. Cualquiera de los tres métodos sirve, sin embargo, conviene tener desacoplado el origen (y posible generación) de los datos de altura de la generación de la malla. La intersección de ambos se debería dar al momento de asignar la coordenada que representa la altura en la malla poligonal, por ejemplo en la dirección "y":

$$y = s \cdot h(i, j)$$

donde s es la **escala vertical** y $h(i, j)$ es la función / método que devuelve el valor de altura en el nodo (i, j). La elección del método para calcular los datos de altura (heightmap) puede afectar la forma que tenga la malla que representa el terreno. Por ejemplo, si se usa una imagen de 8 bits, la misma tiene un único canal que sólo puede representar 256 alturas distintas; si la diferencia máxima de alturas es de mil metros, el menor desnivel que se podrá representar es de casi cuatro metros y puede llegar a verse con aspecto "escalonado".

Fuente de alturas para "Terreno procedural"

Leer un archivo para obtener los datos de altura es una opción, pero para empezar conviene generar el relieve con **ruido procedural**: no hay que conseguir ni cargar ningún dato, el relieve no se repite, y la fuente de alturas se simplifica con una llamada a una función.

Un módulo que provee funciones para generar ruido aleatorio es `FastNoiseLite` (https://github.com/Auburn/FastNoiseLite). Tiene la ventaja de que está contenido en un único archivo de cabecera: se copia al proyecto y se incluye, no hay nada que compilar ni enlazar (linkear) aparte.

```
#include "FastNoiseLite.h"

// Fuente de alturas por ruido procedural. Devuelve la altura YA ESCALADA, en
// las mismas unidades que el resto de la escena.
class Alturas {
    FastNoiseLite ruido_;
    float amplitud_;                  // desnivel maximo, en unidades de escena
public:
    Alturas(int semilla, float frecuencia, float amplitud)
        : amplitud_(amplitud)
    {
        ruido_.SetSeed(semilla);
        ruido_.SetNoiseType(FastNoiseLite::NoiseType_Perlin);
        ruido_.SetFractalType(FastNoiseLite::FractalType_FBm);  // suma de octavas
        ruido_.SetFractalOctaves(5);
        ruido_.SetFrequency(frecuencia);
    }
    // GetNoise devuelve aprox. [-1, 1]: la amplitud lo lleva a escala de la escena.
    float operator()(unsigned i, unsigned j) const {
        return amplitud_ * ruido_.GetNoise(float(j), float(i));
    }
};
```

La clase del ejemplo anterior representa la llamada `altura(i,j)` que se usa en el bucle de la Parte 1 y desacopla la generación de la geometría de la lectura del relieve, permitiendo reemplazarla por otras fuentes más adelante. Se pueden obtener distintos tipos de relieve modificando los parámetros de **frecuencia** —cuántos nodos abarca una característica grande del terreno; con 0,01 abarca unos cien nodos— y **amplitud** —que es la escala vertical s de la explicación del modelo de alturas—. Las octavas agregan detalle fino encima del relieve grande y la semilla sirve para poder repetir el mismo terreno en sesiones diferentes.

Conviene tener presente que **la amplitud que se pide no es la que se obtiene**: al sumar octavas la salida queda normalizada y en la práctica usa alrededor de **un tercio** del rango nominal —pedir ± 300 [m] da un relieve de unos ± 105 [m]—. Es mejor imprimir el máximo y el mínimo una vez, al generar la malla, que ajustar la amplitud por tanteos.

La escala de la escena

Al agregar un objeto nuevo a la escena (el terreno) hay que ajustar el tamaño del mismo para que sea coherente con el tamaño del resto de los objetos presentes (aeronave). Por lo tanto hay que fijar un valor de *escala global* en donde se debe elegir la relación que existe entre las unidades de medida de los objetos y las unidades de la escena.

Así por ejemplo, si la aeronave ahora tiene un largo total de 4 unidades (en OpenGL) y el largo real es 8 [m], entonces quiere decir que cada unidad de medida del modelo representa 2 [m]. Si usamos esta relación al crear los vértices de la malla del terreno, cada celda tendrá 2 [m] de lado y, asumiendo $N = 200$, el terreno tendrá un tamaño total de 400×400 [m] que puede no ser adecuado para la escena que se está queriendo representar (entorno de vuelo). Para esta escena sería más razonable elegir para la separación de los vértices de la malla del terreno una distancia del orden de 25 [m] que genera una superficie de 5000×5000 [m] y ajustar la escala del modelo para reducirlo a 0,32 unidades de dibujo de largo (8 [m]/25 [m]). Este ejemplo no es una definición de lo que hay que hacer, sino que tiene como objetivo mostrar que ahora los tamaños de los objetos en la escena dependen unos de otros.

Adecuar el tamaño de la malla del terreno a la **escala** de la escena requiere analizar tres cuestiones distintas:

- *separación entre vértices*: es el ancho de la celda en unidades de la escena y fija el detalle del relieve (separación pequeña → detalle alto, pero extensión pequeña; separación grande → detalle bajo, pero extensión grande).
- *escala vertical*: es el ajuste de los datos de altura a unidades de la escena y fija la amplitud del relieve.

- *tamaño de la grilla*: es la cantidad de celdas (N) y fija la extensión del mundo.

Por ejemplo, si los datos de altura se generan con una función de ruido y el terreno sale con muchos picos o muy planchado, lo más probable es que el error esté en la escala vertical y no en los parámetros de la función.

El tamaño del mundo y los límites

Una consideración importante en el tipo de escena que se está tratando de modelar es el tamaño que debe tener el mundo. El modelo de la aeronave que resuelve el FDM (entrenador con motor a reacción) tiene una capacidad de volar a 150 m/s por lo que esta velocidad es un valor posible que se va a obtener de la salida del FDM. Volando a esa velocidad, la aeronave recorre 2 kilómetros en 13 segundos y 20 kilómetros en poco más de dos minutos, por lo que rápidamente se van a alcanzar los límites del terreno.

Agrandar la malla no resuelve el problema, sólo lo posterga: el borde sigue estando y se alcanza igual, solamente que se demora más tiempo. Conviene notar que la memoria no es la restricción: crece con la cantidad de celdas, no con la extensión del mundo, así que las dos últimas filas cubren 20 y 40 kilómetros con la misma memoria. Estirar la separación entre vértices duplica el mundo sin incrementar el tamaño, sin embargo, lo que se pierde es **nivel de detalle**. Es una decisión a considerar al momento de dimensionar el mundo que va a ser procesado.

celdas	separación	lado	memoria	tiempo en cruzarlo
200	10 m	2,0 km	1,8 MiB	13 s
400	25 m	10,0 km	7,3 MiB	1 min 7 s
800	25 m	20,0 km	29,3 MiB	2 min 13 s
800	50 m	40,0 km	29,3 MiB	4 min 27 s

Memoria de la malla *indexada* considerando vértices de 24 bytes —posición y normal— e índices de 32 bits.

Entonces, hay que elegir un tamaño y un comportamiento para el borde y registrarlo como decisión. Las siguientes pueden ser dos opciones posibles (no son las únicas respuestas a este problema):

- **repetir la misma malla** con distintas matrices de modelo —la misma malla dibujada varias veces. Usando un arreglo de 3×3 y una malla de 20 [km] se obtienen 60 [km] de lado con la misma memoria y nueve llamadas de dibujo (`glDraw*`).
- **tratar el borde como una regla del simulador**: aviso y retorno, o fin del vuelo. Esta opción no es una limitación técnica disimulada: un simulador que debe seguir un circuito de vuelo (acrobacias por ejemplo) requiere que la aeronave permanezca en una región determinada de la escena.

Las normales

Hay que prepararlas en este práctico aunque no se usan todavía: el terreno se dibuja ahora con un color plano o como malla de alambre (recomendado) y las normales se utilizan, recién, cuando se incorporen los modelos de iluminación.

La normal de un vértice interior se obtiene considerando la derivada de la superficie que representa el terreno en el punto que corresponde al vértice: para $y = f(x, z)$, la normal hacia arriba viene dada por $\hat{n}(x, z) = (-\partial f/\partial x,\ 1,\ -\partial f/\partial z)$. Como la superficie del terreno está descrita por medio de valores discretos (la posición (x, z) del vértice y la altura correspondiente) hay que determinar las dos derivadas parciales que aparecen por medio de un esquema de **derivada numérica** por diferencias centradas que estiman las dos derivadas usando los datos de los nodos vecinos.

```
// Normal hacia arriba de un vertice interior, por diferencias centradas.
// altura(i,j) es la misma fuente de la Parte 1: devuelve la altura ya escalada,
// en unidades de la escena, igual que la coordenada y del vertice.
glm::vec3 normal(unsigned i, unsigned j, float d)
{
    return glm::normalize(glm::vec3(
        altura(i, j-1) - altura(i, j+1),      // -df/dx, por 2d
        2.0f * d,
        altura(i-1, j) - altura(i+1, j)       // -df/dz, por 2d
    ));
}
```

En el borde de la grilla no hay cuatro vecinos: ahí se usa una diferencia de un solo lado, o se repite la normal del vértice interior contiguo. Las dos opciones son aceptables y la diferencia no se ve; lo que no se puede es dejar la normal en cero, porque normalizar un vector nulo no está definido.

Nota:
Hay un segundo método para obtener la normal de sombreado en un vértice compartido: promediar las normales de los triángulos que comparten el vértice —los seis de la Parte 1—.

Los dos métodos son válidos y en una grilla regular dan resultados similares. Cuánto se parecen depende del **nivel de granularidad** del relieve frente a la separación entre vértices: sobre una superficie suave son indistinguibles y usando el ruido de cinco octavas del ejemplo anterior difieren alrededor de 1° en el peor vértice, que no se alcanza a notar cuando se aplica iluminación. Lo que sí cambia es el costo computacional y el alcance. La estimación por diferencias centradas requiere cuatro lecturas de altura, no necesita los triángulos y no cambia si se altera la diagonal de las celdas. El promedio de normales de las caras es un método **general** pero exige una segunda pasada sobre la lista de índices para determinar las normales de los triángulos (más costoso). Sin embargo, es la estrategia que se aplica a una malla general que no disponga de una función de alturas como es el caso del terreno (un modelo importado, por ejemplo).

Parte 3 — Eliminación de superficies ocultas

La aplicación práctica de la teoría de eliminación de superficies ocultas no presenta mayores problemas, ya que en OpenGL se reduce a activar dos funcionalidades que están asociadas a la visibilidad de los polígonos: *test de profundidad* (`DEPTH_TEST`) y *recorte de cara trasera* (`CULL_FACE`). Estas dos funcionalidades vienen desactivadas por defecto (no se determina profundidad de los objetos al digitalizarlos y no se aplica recorte de cara trasera).

```
glEnable(GL_DEPTH_TEST);   // test de profundidad: z-buffer
glEnable(GL_CULL_FACE);    // recorte de cara trasera
```

El *test de profundidad* no suele dar problemas: resuelve la visibilidad por fragmento comparando profundidades contra el *z-buffer*, así el resultado no depende del orden en que se dibujen los polígonos —que es justamente la ventaja del método—. El *recorte de cara trasera* sí suele traer problemas de visualización al activarlo: es bastante probable que desaparezcan caras que se deberían ver —por ejemplo, puede dejar de verse el terreno entero o puede verse por zonas o parches— sin que haya ningún error de compilación y ningún mensaje de OpenGL.

Esto se debe a que OpenGL **no usa el atributo normal** para decidir qué cara descarta. En cambio, toma el signo del área del triángulo ya proyectado en pantalla: si sus tres vértices quedan en **sentido antihorario** lo considera *cara frontal*. Es decir, que decide por *el orden en que se listaron los vértices* para dibujar. Por eso hay que tener en cuenta el orden de los vértices al generar las listas para enviar al GPU si se va a utilizar la optimización de recorte de cara trasera.

Cómo diagnosticar problemas de recorte de cara trasera
Cada paso descarta una familia entera de problemas y se pueden verificar fácilmente cambiando una línea en el código:

1. `glPolygonMode(GL_FRONT_AND_BACK, GL_LINE)`: ¿el objeto aparece como "malla de alambre"? Si no aparece, el problema **no es el recorte**: es la geometría, la cámara o la matriz modelo.
2. `glDisable(GL_CULL_FACE)`: ¿vuelven a ser visibles las caras? Si vuelven, es un problema de **orden en los vértices (winding)**, no de geometría.
3. `glFrontFace(GL_CW)`: ¿vuelven a ser visibles las caras? Si vuelven, es un problema de **inversión del orden de todos los vértices**: todo el bucle de ordenamiento está al revés.
4. ¿Los agujeros son **regulares** con forma de tablero? Entonces es **inconsistencia**: una parte del bucle ordena distinto de la otra.

La corrección se hace siempre **en el generador de la malla**, no en el estado de OpenGL para dibujar: `glFrontFace` no reordena los vértices, sólo cambia qué enrollado se considera frontal. Usarlo para que el modelo "se vea bien" deja el problema dentro del generador, y vuelve a aparecer con la primera malla nueva que no tenga el mismo ordenamiento.

Opcional

- **Terreno ilimitado usando una ventana que se desplaza.** El heightmap cubre un mundo mayor que el que se visualiza y es estático; la malla es una ventana que sigue a la aeronave. Los índices y la topología se construyen una sola vez y no se vuelven a tocar: lo único que se recalcula son las alturas al cruzar una celda. Una cuestión a controlar: si la ventana sigue a la aeronave de forma continua el terreno **repta**: los vértices caen *entre* nodos del heightmap, hay que interpolar, y la triangulación cambia de un cuadro al siguiente, así que la superficie parece deslizarse. La ventana tiene que moverse **a saltos**, en múltiplos de la separación entre vértices: con el origen redondeado cada vértice vuelve a caer sobre un nodo y la superficie dibujada es la misma esté donde esté la ventana.
- **Leer el relieve de un archivo** en lugar de generarlo: una imagen en escala de grises, o un archivo de texto con datos topográficos reales. Si la fuente de alturas quedó desacoplada de la generación de la malla como se propone en la Parte 2, aplicar este método se reduce a cambiar una sola clase.
- **Índices con representación de 16 bits.** Considerando 40.401 vértices, los índices se pueden representar mediante 16 bits, porque el máximo es 65.535. Conviene preguntarse qué pasa cuando la grilla aumenta a 256×256.

Algunas cuestiones a pensar

- El bucle de vértices va hasta `<= N` y el de celdas hasta `< N`. ¿Qué se ve exactamente si se escriben los dos iguales? Conviene probarlo a propósito para poder diagnosticar en el futuro.
- Una mejora razonable del terreno es **alternar la diagonal** de las celdas, para que el relieve no quede peinado en una sola dirección. Hecha sin cuidado deja la mitad de los triángulos al revés y los agujeros salen formando un tablero de ajedrez. ¿Por qué justo la mitad y por qué en ese patrón?

Cómo se sabe que está bien

- Con el recorte de cara trasera **activado**, el terreno se ve completo desde arriba: ni desaparece entero, ni le faltan parches.
- En modo alambre se cuentan $N + 1$ líneas por lado, no N, y el último tramo del terreno está dibujado.
- El relieve **coincide con el dato**: una zona alta del heightmap es una zona alta del terreno, y no aparece espejada ni rotada.
- Mirando el terreno desde abajo, con el recorte activado, **no se ve**: es la comprobación de que las caras están orientadas hacia arriba y no al revés.
- La aeronave llega al borde del mundo y pasa lo que se decidió que pasara, y eso está escrito.

Como en los prácticos anteriores, las interfaces mostradas son un **punto de partida y no el molde**. Las decisiones de adentro son de cada uno y hay que poder explicarlas.

El **checkpoint del proyecto** está planificado para el **viernes 27 de noviembre**. Hay que ir preparando las funcionalidades mínimas del proyecto.